II laboratorinis darbas

Paprastosios diferencialinės lygties pradinio uždavinio skaitinis sprendimas
Variantas Nr. 9

Edvin Katenel, 1 grupė

Užduoties formulavimas

Išspręskite Koši uždavinį

$$\begin{cases} \dfrac{du}{dx} = f(x,u) = x^2\cos(u-x), \\ u(0) = u_0, \end{cases} \qquad 0 \le x \le 1,$$

taikydami 4-pakopį Rungės–Kuto metodą ir antrąjį metodą Nr. 3 – dvipakopį Rungės–Kuto metodą, kai $\sigma = 0.5$. Pasirinkta pradinė sąlyga $u(0) = 0$, skaičiuota su žingsniais $\tau = 0.1$ ir $\tau = 0.05$. Paklaidą intervale $(0, 1]$ įvertinkite Rungės metodu. Uždavinį taip pat išspręskite kompiuterine funkcija, kurioje naudojamas Rungės–Kuto metodas, ir grafiškai palyginkite rezultatus: (a) metodas tas pats, skiriasi žingsnis; (b) žingsnis tas pats, skiriasi metodas.

Programa parašyta `Python` kalba (`NumPy`, `Matplotlib`). Kaip kompiuterinė funkcija naudota `solve_ivp` iš bibliotekos `scipy.integrate` su metodu `RK45` (Dormando–Princo įterptinis Rungės–Kuto metodas su automatiniu žingsnio parinkimu), tolerancijos `rtol` $= 10^{-12}$, `atol` $= 10^{-14}$. Lygtis nėra nei tiesinė, nei su atskiriamaisiais kintamaisiais, jos analitinio sprendinio neturime, todėl `solve_ivp` sprendinys laikomas etaloniniu $u(x)$. Programos kodas pateiktas priede.

Tinklas: $x_i = i\tau$, $i = 0, 1, \ldots, N$, $N = 1/\tau$; $y_i \approx u(x_i)$, $y_0 = u_0$.

1 Taikyti metodai

1.1 4-pakopis Rungės–Kuto metodas ($p = 4$)

$$k_1 = f(x_i,\, y_i), \qquad k_2 = f\Big(x_i + \tfrac{\tau}{2},\, y_i + \tfrac{\tau}{2}k_1\Big),$$
$$k_3 = f\Big(x_i + \tfrac{\tau}{2},\, y_i + \tfrac{\tau}{2}k_2\Big), \quad k_4 = f(x_i + \tau,\, y_i + \tau k_3),$$
$$y_{i+1} = y_i + \frac{\tau}{6}\,(k_1 + 2k_2 + 2k_3 + k_4).$$

Geometrinė interpretacija: krypties koeficientas apskaičiuojamas keturiuose taškuose (intervalo pradžioje, du kartus viduryje ir gale), o žingsnis daromas pagal jų svertinį vidurkį su svoriais $1 : 2 : 2 : 1$ (kaip Simpsono formulėje). Lokalioji paklaida $O(\tau^5)$, globalioji – $O(\tau^4)$.

1.2 Dvipakopis Rungės–Kuto metodas, $\sigma = 0.5$ $(p = 2)$

Bendroji dvipakopių metodų šeima:

$$y_{i+1} = y_i + \tau\Big[(1-\sigma)k_1 + \sigma k_2\Big], \qquad k_1 = f(x_i, y_i), \quad k_2 = f\Big(x_i + \tfrac{\tau}{2\sigma},\, y_i + \tfrac{\tau}{2\sigma}k_1\Big).$$

Kai $\sigma = 0.5$, gauname **prediktoriaus–korektoriaus (Hojno) metodą**:

$$\tilde{y}_{i+1} = y_i + \tau f(x_i, y_i) \quad \text{(prediktorius – Eulerio žingsnis)},$$

$$y_{i+1} = y_i + \frac{\tau}{2}\Big[f(x_i, y_i) + f(x_{i+1}, \tilde{y}_{i+1})\Big] \quad \text{(korektorius – trapecijų formulė)}.$$

Geometrinė interpretacija: pirmiausia Eulerio metodu (liestine) „nuspėjamas“ taškas intervalo gale, tada žingsnis daromas pagal krypties koeficientų intervalo pradžioje ir gale vidurkį. Globalioji paklaida $O(\tau^2)$.

1.3 Paklaidos įvertis Rungės metodu

$$|u(T) - y_\tau| \approx \frac{|y_{2\tau} - y_\tau|}{2^p - 1},$$

čia y_τ – sprendinys su žingsniu $\tau = 0.05$, $y_{2\tau}$ – su žingsniu $2\tau = 0.1$, p – metodo tikslumo eilė ($2^p - 1 = 15$ RK4 metodui ir $2^p - 1 = 3$ dvipakopiam metodui). Įvertis skaičiuotas visuose bendruose tinklų taškuose $T = 0.1, 0.2, \ldots, 1$.

2 Rezultatai

1 lentelė. Skaitiniai sprendiniai tinklo taškuose, $u(0) = 0$.

	4-pakopis RK		dvipakopis RK, $\sigma = 0.5$		
x	$\tau = 0.1$	$\tau = 0.05$	$\tau = 0.1$	$\tau = 0.05$	`solve_ivp`
0.0	0.00000000	0.00000000	0.00000000	0.00000000	0.00000000
0.1	0.00033230	0.00033234	0.00049750	0.00037360	0.00033234
0.2	0.00263526	0.00263534	0.00295575	0.00271542	0.00263534
0.3	0.00876816	0.00876827	0.00922511	0.00888257	0.00876827
0.4	0.02039126	0.02039138	0.02095912	0.02053374	0.02039139
0.5	0.03891923	0.03891935	0.03956866	0.03908273	0.03891936
0.6	0.06552296	0.06552307	0.06622384	0.06570032	0.06552308
0.7	0.10117819	0.10117829	0.10190202	0.10136268	0.10117830
0.8	0.14675438	0.14675445	0.14747571	0.14694005	0.14675445
0.9	0.20313742	0.20313747	0.20383384	0.20331910	0.20313748
1.0	0.27138395	0.27138399	0.27203432	0.27155677	0.27138400

2 lentelė. Paklaidos įvertis Rungės metodu ($\tau = 0.05$, $2\tau = 0.1$) ir tikroji paklaida $|u(x) - y_\tau|$, kai $u(x)$ – `solve_ivp` sprendinys.

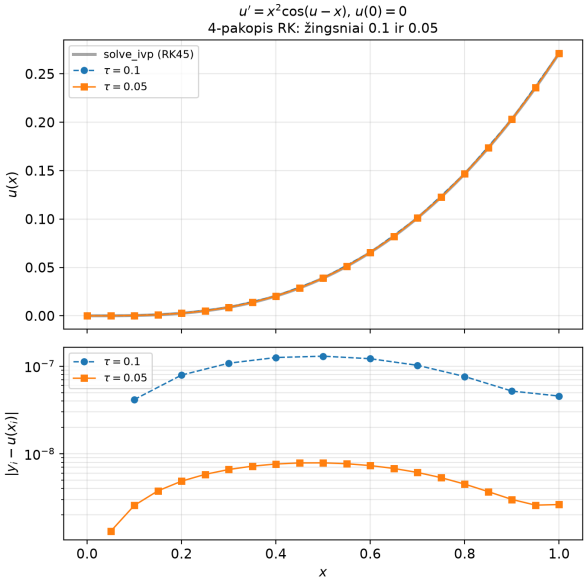
x	4-pakopis RK ($p = 4$) Rungės įvertis	4-pakopis RK ($p = 4$) tikroji paklaida	dvipakopis RK, $\sigma = 0.5$ ($p = 2$) Rungės įvertis	dvipakopis RK, $\sigma = 0.5$ ($p = 2$) tikroji paklaida
0.1	$2.60 \cdot 10^{-9}$	$2.57 \cdot 10^{-9}$	$4.13 \cdot 10^{-5}$	$4.13 \cdot 10^{-5}$
0.2	$4.95 \cdot 10^{-9}$	$4.86 \cdot 10^{-9}$	$8.01 \cdot 10^{-5}$	$8.01 \cdot 10^{-5}$
0.3	$6.76 \cdot 10^{-9}$	$6.60 \cdot 10^{-9}$	$1.14 \cdot 10^{-4}$	$1.14 \cdot 10^{-4}$
0.4	$7.86 \cdot 10^{-9}$	$7.63 \cdot 10^{-9}$	$1.42 \cdot 10^{-4}$	$1.42 \cdot 10^{-4}$
0.5	$8.14 \cdot 10^{-9}$	$7.87 \cdot 10^{-9}$	$1.62 \cdot 10^{-4}$	$1.63 \cdot 10^{-4}$
0.6	$7.62 \cdot 10^{-9}$	$7.32 \cdot 10^{-9}$	$1.75 \cdot 10^{-4}$	$1.77 \cdot 10^{-4}$
0.7	$6.41 \cdot 10^{-9}$	$6.11 \cdot 10^{-9}$	$1.80 \cdot 10^{-4}$	$1.84 \cdot 10^{-4}$
0.8	$4.78 \cdot 10^{-9}$	$4.50 \cdot 10^{-9}$	$1.79 \cdot 10^{-4}$	$1.86 \cdot 10^{-4}$
0.9	$3.26 \cdot 10^{-9}$	$3.01 \cdot 10^{-9}$	$1.72 \cdot 10^{-4}$	$1.82 \cdot 10^{-4}$
1.0	$2.86 \cdot 10^{-9}$	$2.63 \cdot 10^{-9}$	$1.59 \cdot 10^{-4}$	$1.73 \cdot 10^{-4}$
$\max_{(0,1]}$	$8.14 \cdot 10^{-9}$	$7.87 \cdot 10^{-9}$	$1.80 \cdot 10^{-4}$	$1.86 \cdot 10^{-4}$

Rungės įvertis labai gerai sutampa su tikrąja paklaida, t. y. juo galima patikimai vertinti tikslumą, nežinant tikslaus sprendinio. Taške $x = 1$ sumažinus žingsnį perpus, RK4 metodo paklaida sumažėjo nuo $4.55 \cdot 10^{-8}$ iki $2.63 \cdot 10^{-9}$ (apie 17 kartų, teoriškai $2^4 = 16$), o dvipakopio metodo – nuo $6.50 \cdot 10^{-4}$ iki $1.73 \cdot 10^{-4}$ (apie 3.8 karto, teoriškai $2^2 = 4$). Tai patvirtina metodų tikslumo eiles $p = 4$ ir $p = 2$.

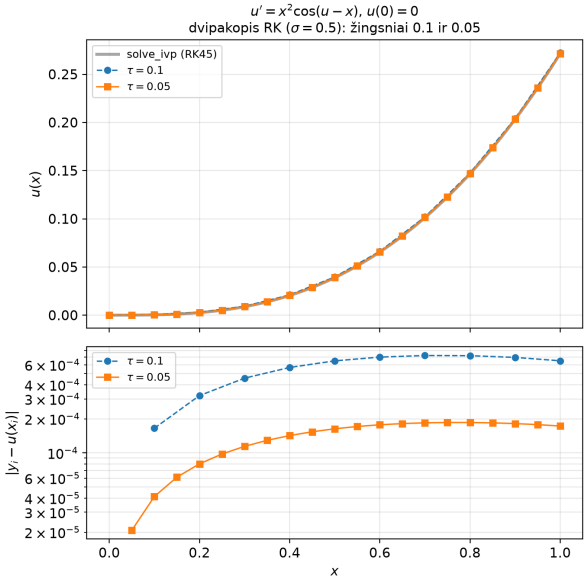
3 Grafinis palyginimas

Kiekviename paveiksle viršuje pavaizduoti sprendiniai (pilka storesnė linija – `solve_ivp`), apačioje – paklaida $|y_i - u(x_i)|$ logaritminėje skalėje, nes viršutiniame grafike kreivės praktiškai sutampa.

3.1 (a) Metodas tas pats, skiriasi žingsnis

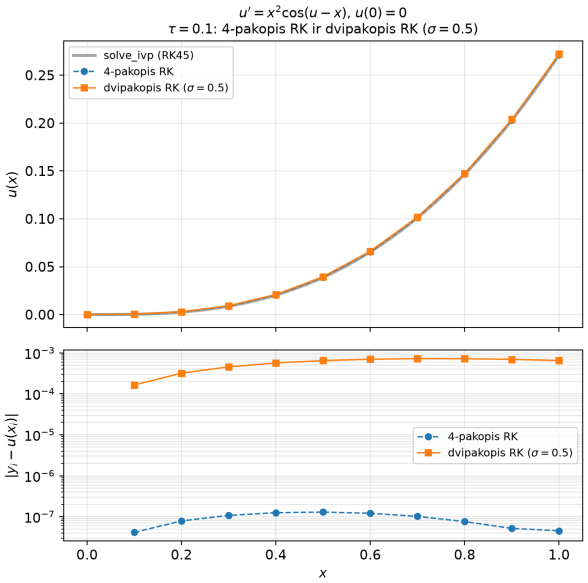


1 pav. 4-pakopis RK metodas su žingsniais $\tau = 0.1$ ir $\tau = 0.05$ bei `solve_ivp` sprendinys. Paveikslėlyje matome, kad visos trys kreivės sutampa, o paklaida su $\tau = 0.1$ yra apie 10^{-7}, su $\tau = 0.05$ – apie 10^{-8}–10^{-9}, t. y. apie 16 kartų mažesnė. Grafikas toks, nes RK4 metodo globalioji paklaida yra $O(\tau^4)$, todėl perpus sumažinus žingsnį paklaida sumažėja $2^4 = 16$ kartų.

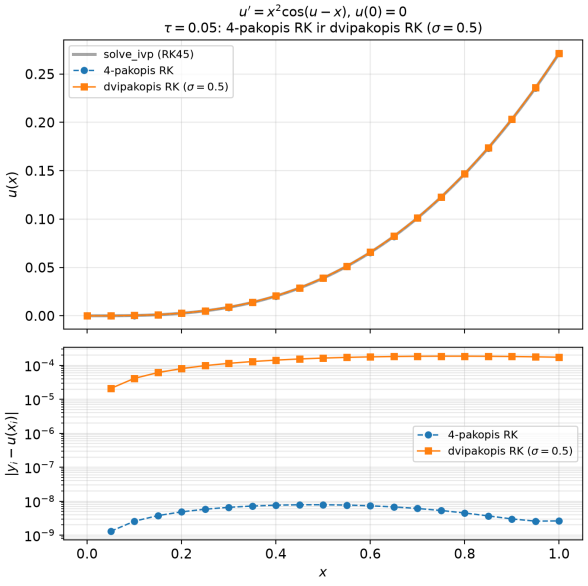


2 pav. Dvipakopis RK metodas ($\sigma = 0.5$) su žingsniais $\tau = 0.1$ ir $\tau = 0.05$ bei `solve_ivp` sprendinys. Paveikslėlyje matome, kad skaitiniai sprendiniai yra šiek tiek virš tikslaus, paklaida auga ir apie $x \approx 0.7$ nusistovi ties $7 \cdot 10^{-4}$ ($\tau = 0.1$) ir $1.8 \cdot 10^{-4}$ ($\tau = 0.05$). Grafikas toks, nes metodo globalioji paklaida yra $O(\tau^2)$ – perpus sumažinus žingsnį paklaida sumažėja apie 4 kartus, o kiekviename žingsnyje susidariusi lokalioji paklaida kaupiasi.

3.2 (b) Žingsnis tas pats, skiriasi metodas



3 pav. Žingsnis $\tau = 0.1$: 4-pakopis ir dvipakopis RK metodai bei `solve_ivp`. Paveikslėlyje matome, kad abu sprendiniai akimi sutampa su `solve_ivp`, tačiau dvipakopio metodo paklaida ($\sim 10^{-4}$–10^{-3}) yra kelis tūkstančius kartų didesnė nei RK4 ($\sim 10^{-7}$). Grafikas toks, nes RK4 metodas kiekviename žingsnyje funkciją f skaičiuoja 4 kartus ir yra ketvirtos eilės, o dvipakopis – tik 2 kartus ir yra antros eilės.



4 pav. Žingsnis $\tau = 0.05$: 4-pakopis ir dvipakopis RK metodai bei `solve_ivp`. Paveikslėlyje matome tą patį vaizdą kaip 3 pav., tik abiejų metodų paklaidos mažesnės, o atotrūkis tarp jų dar didesnis (dešimtys tūkstančių kartų). Grafikas toks, nes mažinant žingsnį aukštesnės eilės metodo paklaida mažėja greičiau (τ^4 prieš τ^2).

Sprendinio kreivė visuose paveiksluose prasideda horizontaliai ir lėtai kyla: kadangi $u(0) = 0$, o $f(0, u) = 0$, pradžioje $u' \approx x^2$, todėl $u \approx x^3/3$; vėliau $\cos(u - x) < 1$ šiek tiek lėtina

augimą, todėl $u(1) \approx 0.2714 < 1/3$.

Išvados

1. Abu metodai ir `solve_ivp` duoda tą patį sprendinį: $u(1) \approx 0.271384$.
2. 4-pakopis RK metodas daug tikslesnis: didžiausia paklaida intervale $(0, 1]$ su $\tau = 0.05$ yra apie $8 \cdot 10^{-9}$, o dvipakopio metodo ($\sigma = 0.5$) – apie $1.9 \cdot 10^{-4}$.
3. Perpus sumažinus žingsnį, paklaida sumažėja apie 2^p kartų (16 ir 4 kartus), kaip ir numato teorija.
4. Rungės metodo įvertis praktiškai sutampa su tikrąja paklaida, todėl yra patikimas būdas vertinti paklaidą, kai tikslus sprendinys nežinomas.

Priedas. Programos kodas

```python
# Edvin Katenel
# II laboratorinis darbas, variantas 9
# Koši uždavinys: u' = f(x, u), u(X0) = U0, X0 <= x <= X1
# Metodai: 4-pakopis Rungės-Kuto (p = 4) ir dvipakopis RK su parametru sigma (p = 2)
import os
import sys
import numpy as np
import matplotlib.pyplot as plt
from scipy.integrate import solve_ivp

sys.stdout.reconfigure(encoding='utf-8')   # lietuviškos raidės Windows terminale
os.chdir(os.path.dirname(os.path.abspath(__file__)))   # paveikslai saugomi šalia programos

#parametrai
def f(x, u):
    return x**2 * np.cos(u - x)

F_TEKSTAS = r"$x^2\cos(u-x)$"
X0, X1 = 0.0, 1.0
U0 = 0.0
TAU = 0.05
SIGMA = 0.5
RODYTI = True
#


def rk4(f, x0, u0, x1, tau):
    """Klasikinis 4-pakopis Rungės-Kuto metodas."""
    n = int(round((x1 - x0) / tau))
    x = x0 + tau * np.arange(n + 1)
    y = np.empty(n + 1); y[0] = u0
    for i in range(n):
        k1 = f(x[i], y[i])
        k2 = f(x[i] + tau / 2, y[i] + tau / 2 * k1)
        k3 = f(x[i] + tau / 2, y[i] + tau / 2 * k2)
        k4 = f(x[i] + tau, y[i] + tau * k3)
        y[i + 1] = y[i] + tau / 6 * (k1 + 2 * k2 + 2 * k3 + k4)
    return x, y


def rk2(f, x0, u0, x1, tau, sigma=0.5):
    """Dvipakopis Rungės-Kuto metodas:
    y_{i+1} = y_i + tau*((1-sigma)*k1 + sigma*k2),
    k1 = f(x_i, y_i),  k2 = f(x_i + a*tau, y_i + a*tau*k1),  a = 1/(2*sigma).
    Kai sigma = 0.5 (a = 1) - Hojno (prediktoriaus-korektoriaus) metodas."""
    a = 1 / (2 * sigma)
    n = int(round((x1 - x0) / tau))
    x = x0 + tau * np.arange(n + 1)
    y = np.empty(n + 1); y[0] = u0
    for i in range(n):
        k1 = f(x[i], y[i])
```

```
        k2 = f(x[i] + a * tau, y[i] + a * tau * k1)
        y[i + 1] = y[i] + tau * ((1 - sigma) * k1 + sigma * k2)
    return x, y


#skaičiavimai
T1, T2 = 2 * TAU, TAU            # žingsniai 2*tau ir tau
assert abs((X1 - X0) / T1 - round((X1 - X0) / T1)) < 1e-9, "intervalo
    ilgis turi dalintis iš 2*TAU"

res = {}
for tau in (T1, T2):
    res[('RK4', tau)] = rk4(f, X0, U0, X1, tau)
    res[('RK2', tau)] = rk2(f, X0, U0, X1, tau, sigma=SIGMA)

#
ref = solve_ivp(f, (X0, X1), [U0], method='RK45', rtol=1e-12, atol=1e
    -14, dense_output=True)
u_ref = lambda x: ref.sol(x)[0]

p = {'RK4': 4, 'RK2': 2}
xg = res[('RK4', T1)][0]

print(f"Reikšmės tinklo taškuose (u({X0:g}) = {U0:g})")
print(f"{'x':>6} | {'RK4 '+str(T1):>12} {'RK4 '+str(T2):>12} | {'RK2 '+
    str(T1):>12} {'RK2 '+str(T2):>12} | {'solve_ivp':>12}")
for k, xv in enumerate(xg):
    print(f"{xv:6.2f} | {res[('RK4',T1)][1][k]:12.8f} {res[('RK4',T2)
        ][1][2*k]:12.8f} | "
          f"{res[('RK2',T1)][1][k]:12.8f} {res[('RK2',T2)][1][2*k]:12.8f
              } | {u_ref(xv):12.8f}")

print(f"\nPaklaidos įvertis Rungės metodu |y_2tau - y_tau| / (2^p - 1),
    tau = {T2:g}")
print(f"{'x':>6} | {'RK4 įvertis':>12} {'RK4 tikroji':>12} | {'RK2 į
    vertis':>12} {'RK2 tikroji':>12}")
runge = {}
for m in ('RK4', 'RK2'):
    y2t = res[(m, T1)][1]            # žingsnis 2*tau
    yt = res[(m, T2)][1][::2]        # žingsnis tau tuose pačiuose taš
        kuose
    runge[m] = (np.abs(y2t - yt) / (2**p[m] - 1), np.abs(u_ref(xg) - yt)
        )
for k in range(1, len(xg)):
    print(f"{xg[k]:6.2f} | {runge['RK4'][0][k]:12.3e} {runge['RK4'][1][k
        ]:12.3e} | "
          f"{runge['RK2'][0][k]:12.3e} {runge['RK2'][1][k]:12.3e}")
for m in ('RK4', 'RK2'):
    print(f"{m}: max Rungės įvertis ({X0:g};{X1:g}] = {runge[m][0][1:].
        max():.3e}, "
          f"max tikroji paklaida = {runge[m][1][1:].max():.3e}")

#
for m in ('RK4', 'RK2'):
    e1 = abs(res[(m, T1)][1][-1] - u_ref(X1))
    e2 = abs(res[(m, T2)][1][-1] - u_ref(X1))
    print(f"{m}: paklaida x={X1:g}: tau={T1:g} -> {e1:.3e}, tau={T2:g}
        -> {e2:.3e}, "
```

```
          f"santykis {e1/e2:.2f} (2^p = {2**p[m]})")

# grafikai
plt.rcParams.update({"font.size": 11})
xx = np.linspace(X0, X1, 400)
names = {'RK4': '4-pakopis RK', 'RK2': rf'dvipakopis RK ($\sigma={SIGMA:g}$)'}


def two_panel(title, curves, fname):
    """Viršuje - sprendiniai, apačioje - |y - u_ref| logaritminėje skalėje."""
    fig, (a1, a2) = plt.subplots(2, 1, figsize=(7, 7), sharex=True,
                                 gridspec_kw={'height_ratios': [3, 2]})
    a1.plot(xx, u_ref(xx), 'k-', lw=2.5, alpha=.35, label='solve_ivp (RK45)')
    for (x, y, lab, st) in curves:
        a1.plot(x, y, st, ms=5, lw=1.2, label=lab)
        a2.semilogy(x[1:], np.abs(y[1:] - u_ref(x[1:])), st, ms=5, lw=1.2, label=lab)
    a1.set_title(f"$u'=${F_TEKSTAS}, $u({X0:g})={U0:g}$\n" + title, fontsize=11)
    a1.set_ylabel('$u(x)$'); a1.grid(alpha=.3); a1.legend(fontsize=9)
    a2.set_xlabel('$x$'); a2.set_ylabel(r'$|y_i-u(x_i)|$'); a2.grid(alpha=.3, which='both')
    a2.legend(fontsize=9)
    fig.tight_layout(); fig.savefig(fname, dpi=200)
    print("Išsaugota:", os.path.abspath(fname))


for m, fn in (('RK4', 'fig_a_rk4.png'), ('RK2', 'fig_a_rk2.png')):
    two_panel(names[m] + f': žingsniai {T1:g} ir {T2:g}',
              [(*res[(m, T1)], rf'$\tau={T1:g}$', 'o--'), (*res[(m, T2)], rf'$\tau={T2:g}$', 's-')], fn)

for tau, fn in ((T1, 'fig_b_2tau.png'), (T2, 'fig_b_tau.png')):
    two_panel(rf'$\tau={tau:g}$: 4-pakopis RK ir dvipakopis RK ($\sigma={SIGMA:g}$)',
              [(*res[('RK4', tau)], names['RK4'], 'o--'), (*res[('RK2', tau)], names['RK2'], 's-')], fn)

if RODYTI:
    plt.show()
```